ANAPHOR COREFERENCE RESOLUTION UPDATED

anaphora_resolution_updated.py [newhorizon...epoint.com]

This program demonstrates **Anaphora Resolution (Pronoun Resolution)** using **spaCy**. It identifies pronouns such as **He, Him, His** and attempts to determine which person they refer to.

---

1. Import spaCy

```
1  import spacy
```

- Imports the spaCy Natural Language Processing library.
- spaCy provides tokenization, POS tagging, Named Entity Recognition (NER), etc.

---

2. Load English Language Model

```
1  nlp = spacy.load("en_core_web_sm")
```

- Loads the pre-trained English model.
- en_core_web_sm contains:
    - Tokenizer
    - POS Tagger
    - Named Entity Recognizer
    - Parser

---

3. Input Text

```
1  text = """
2  Rahul is a student.
3  He studies computer science.
4  Rahul likes Python.
5  He wants to become a software engineer.
6  """
```

Sample paragraph containing:

- Proper noun: **Rahul**
- Pronouns: **He**

Goal:

```
1  He → Rahul
```

4. Process the Text

```
doc = nlp(text)
```

- Converts raw text into a spaCy document object.
- spaCy automatically:
  - Splits sentences
  - Splits words
  - Identifies parts of speech
  - Finds entities

5. Display Heading

```
print("ANAPHORA AND COREFERENCE RESOLUTION")
print("---------------------------------")
```

Output:

```
ANAPHORA AND COREFERENCE RESOLUTION
---------------------------------
```

6. Create Empty List for Antecedents

```
antecedents = []
```

Antecedent means:

> The noun to which a pronoun refers.

Example:

```
Rahul is a student.
He studies computer science.

Antecedent = Rahul
Pronoun = He
```

7. Extract PERSON Entities

```
for ent in doc.ents:
    if ent.label_ == "PERSON" and ent.text not in antecedents:
        antecedents.append(ent.text)
```

How it works

spaCy's NER identifies names.

Example:

Word	Entity Type
Rahul	PERSON

So:

```
antecedents = ["Rahul"]
```

8. Find Additional Proper Nouns

```
for token in doc:
```

Loops through every word.

Example words:

```
Rahul
is
a
student
He
studies
...
```

Check Proper Nouns

```
if token.pos_ == "PROPN"
```

PROPN = Proper Noun

Examples:

```
Rahul
India
Microsoft
```

Avoid Duplicates

```
and token.text not in antecedents
```

Prevents adding same name twice.

Ignore Python

```
1  and token.text not in ["Python"]
```

Because Python is a programming language, not a person.

Add Name

```
1  antecedents.append(token.text)
```

Result:

```
1  ["Rahul"]
```

---

9. Print Antecedents

```
1  for ant in antecedents:
2      print(f"   - {ant}")
```

Output:

```
1  Potential Antecedents:
2     - Rahul
```

---

10. Start Rule-Based Resolution

```
1  print("\nPronoun-Antecedent Resolution:")
```

Displays section heading.

---

11. Set Initial Antecedent

```
1  current_antecedent = antecedents[0] if antecedents else
"Unknown"
```

Meaning

If antecedent exists:

```
1  current_antecedent = "Rahul"
```

Otherwise:

```
1  current_antecedent = "Unknown"
```

---

12. Process One Sentence at a Time

```
for sent in doc.sents:
```

Sentences become:

```
Rahul is a student.
```

```
He studies computer science.
```

```
Rahul likes Python.
```

```
He wants to become a software engineer.
```

13. Process Every Word

```
for token in sent:
```

Visits each token inside the sentence.

Example:

```
Rahul
is
a
student
.
```

14. Update Current Antecedent

```
if token.text in antecedents:
    current_antecedent = token.text
```

When the program sees:

```
Rahul
```

it updates:

```
current_antecedent = "Rahul"
```

This helps track the most recently mentioned person.

15. Detect Pronouns

```
if token.pos_ == "PRON"
```

Checks whether the word is a pronoun.

Examples:

```
He
```

```
She
They
His
Him
```

16. Check Masculine Pronouns

```
and token.text.lower() in
["he", "him", "his", "himself"]
```

Converts to lowercase and checks whether pronoun is:

```
he
him
his
himself
```

17. Resolve the Pronoun

```
print(
f"Resolved Pronoun: '{token.text}' -> '{current_antecedent}'"
)
```

Example:

```
He -> Rahul
```

18. Full Output Line

```
print(
f"   - Resolved Pronoun: '{token.text}' ->
'{current_antecedent}'
in sentence: \"{sent.text.strip()}\""
)
```

Example Output:

```
Resolved Pronoun: 'He' -> 'Rahul'
in sentence: "He studies computer science."
```

19. Final Message

```
print("\nCoreference Resolution Completed.")
```

Output:

```
Coreference Resolution Completed.
```

Complete Execution Flow

Input

```
Rahul is a student.
He studies computer science.
Rahul likes Python.
He wants to become a software engineer.
```

Detected Antecedent

```
Rahul
```

Detected Pronouns

```
He
He
```

Resolution

```
He -> Rahul
He -> Rahul
```

Final Output

```
ANAPHORA AND COREFERENCE RESOLUTION
----------------------------------

Potential Antecedents:
   - Rahul

Pronoun-Antecedent Resolution:
   - Resolved Pronoun: 'He' -> 'Rahul'
     in sentence: "He studies computer science."

    - Resolved Pronoun: 'He' -> 'Rahul'
      in sentence: "He wants to become a software engineer."

 Coreference Resolution Completed.
```

Viva Questions

1. What is an anaphora in NLP?

2. What is the difference between anaphora resolution and coreference resolution?
3. What is an antecedent?
4. What is a pronoun in NLP?
5. What does en_core_web_sm provide?
6. What is Named Entity Recognition (NER)?
7. Why is Rahul identified as a PERSON entity?
8. What does token.pos_ return?
9. Why is "Python" excluded from antecedents?
10. What are the limitations of this rule-based approach?

anaphora_resolution_updated.py demonstrates a simple **rule-based Anaphora Resolution system** where masculine pronouns (**He, Him, His**) are mapped to the most recently mentioned person (**Rahul**) using spaCy's NER and POS tagging features. [newhorizon...epoint.com]